

```

In [3]: # Single line comment
letter = 'P' # A string could be a single character or a bunch of te
print(letter) # P
print(len(letter)) # 1
greeting = 'Hello, World!' # String could be a single or double quote, "Hello, Wor
print(greeting) # Hello, World!
print(len(greeting)) # 13
sentence = "I hope you are enjoying 30 days of python challenge"
print(sentence)

# Multiline String
multiline_string = '''I am a teacher and enjoy teaching.
I didn't find anything as rewarding as empowering people.
That is why I created 30 days of python.'''
print(multiline_string)
# Another way of doing the same thing
multiline_string = """I am a teacher and enjoy teaching.
I didn't find anything as rewarding as empowering people.
That is why I created 30 days of python."""
print(multiline_string)

# String Concatenation
first_name = 'Asabeneh'
last_name = 'Yetayeh'
space = ' '
full_name = first_name + space + last_name
print(full_name) # Asabeneh Yetayeh
# Checking length of a string using len() builtin function
print(len(first_name)) # 8
print(len(last_name)) # 7
print(len(first_name) > len(last_name)) # True
print(len(full_name)) # 15

#### Unpacking characters
language = 'Python'
a,b,c,d,e,f = language # unpacking sequence characters into variables
print(a) # P
print(b) # y
print(c) # t
print(d) # h
print(e) # o
print(f) # n

# Accessing characters in strings by index
language = 'Python'
first_letter = language[0]
print(first_letter) # P
second_letter = language[1]
print(second_letter) # y
last_index = len(language) - 1
last_letter = language[last_index]
print(last_letter) # n

# If we want to start from right end we can use negative indexing. -1 is the last i

```

```

language = 'Python'
last_letter = language[-1]
print(last_letter) # n
second_last = language[-2]
print(second_last) # o

# Slicing

language = 'Python'
first_three = language[0:3] # starts at zero index and up to 3 but not include 3
last_three = language[3:6]
print(last_three) # hon
# Another way
last_three = language[-3:]
print(last_three) # hon
last_three = language[3:]
print(last_three) # hon

# Skipping character while splitting Python strings
language = 'Python'
pto = language[0:6:2] #
print(pto) # pto

# Escape sequence
print('I hope every one enjoying the python challenge.\nDo you ?') # Line break
print('Days\tTopics\tExercises')
print('Day 1\t3\t5')
print('Day 2\t3\t5')
print('Day 3\t3\t5')
print('Day 4\t3\t5')
print('This is a back slash symbol (\\)') # To write a back slash
print('In every programming language it starts with \"Hello, World!\"')

## String Methods
# capitalize(): Converts the first character the string to Capital Letter

challenge = 'thirty days of python'
print(challenge.capitalize()) # 'Thirty days of python'

# count(): returns occurrences of substring in string, count(substring, start=.., end=...)
challenge = 'thirty days of python'
print(challenge.count('y')) # 3
print(challenge.count('y', 7, 14)) # 1
print(challenge.count('th')) # 2

# endswith(): Checks if a string ends with a specified ending

challenge = 'thirty days of python'
print(challenge.endswith('on')) # True
print(challenge.endswith('tion')) # False

# expandtabs(): Replaces tab character with spaces, default tab size is 8. It takes
challenge = 'thirty\tdays\ttof\tpython'
print(challenge.expandtabs()) # 'thirty  days    of      python'

```

```

print(challenge.expandtabs(10)) # 'thirty    days    of    python'

# find(): Returns the index of first occurrence of substring

challenge = 'thirty days of python'
print(challenge.find('y')) # 5
print(challenge.find('th')) # 0

# format()    formats string into nicer output
first_name = 'Asabeneh'
last_name = 'Yetayeh'
job = 'teacher'
country = 'Finland'
sentence = 'I am {} {}. I am a {}. I live in {}.'.format(first_name, last_name, job)
print(sentence) # I am Asabeneh Yetayeh. I am a teacher. I live in Finland.

radius = 10
pi = 3.14
area = pi # radius ** 2
result = 'The area of circle with {} is {}'.format(str(radius), str(area))
print(result) # The area of circle with 10 is 314.0

# index(): Returns the index of substring
challenge = 'thirty days of python'
print(challenge.find('y')) # 5
print(challenge.find('th')) # 0

# isalnum(): Checks alphanumeric character

challenge = 'ThirtyDaysPython'
print(challenge.isalnum()) # True

challenge = '30DaysPython'
print(challenge.isalnum()) # True

challenge = 'thirty days of python'
print(challenge.isalnum()) # False

challenge = 'thirty days of python 2019'
print(challenge.isalnum()) # False

# isalpha(): Checks if all characters are alphabets

challenge = 'thirty days of python'
print(challenge.isalpha()) # True
num = '123'
print(num.isalpha()) # False

# isdecimal(): Checks Decimal Characters

challenge = 'thirty days of python'
print(challenge.find('y')) # 5
print(challenge.find('th')) # 0

# isdigit(): Checks Digit Characters

```

```
challenge = 'Thirty'
print(challenge.isdigit()) # False

# isdecimal():Checks decimal characters

num = '10'
print(num.isdecimal()) # True
num = '10.5'
print(num.isdecimal()) # False

# isidentifier():Checks for valid identifier means it check if a string is a valid

challenge = '30DaysOfPython'
print(challenge.isidentifier()) # False, because it starts with a number
challenge = 'thirty_days_of_python'
print(challenge.isidentifier()) # True

# islower():Checks if all alphabets in a string are lowercase

challenge = 'thirty days of python'
print(challenge.islower()) # True
challenge = 'Thirty days of python'
print(challenge.islower()) # False

# isupper(): returns if all characters are uppercase characters

challenge = 'thirty days of python'
print(challenge.isupper()) # False
challenge = 'THIRTY DAYS OF PYTHON'
print(challenge.isupper()) # True

# isnumeric():Checks numeric characters

num = '10'
print(num.isnumeric()) # True
print('ten'.isnumeric()) # False

# join(): Returns a concatenated string

web_tech = ['HTML', 'CSS', 'JavaScript', 'React']
result = '#, '.join(web_tech)
print(result) # 'HTML# CSS# JavaScript# React'

# strip(): Removes both Leading and trailing characters

challenge = ' thirty days of python '
print(challenge.strip('y')) # 5

# replace(): Replaces substring inside

challenge = 'thirty days of python'
print(challenge.replace('python', 'coding')) # 'thirty days of coding'
```

```
# split(): Splits String from Left

challenge = 'thirty days of python'
print(challenge.split()) # ['thirty', 'days', 'of', 'python']

# title(): Returns a Title Cased String

challenge = 'thirty days of python'
print(challenge.title()) # Thirty Days Of Python

# swapcase(): Checks if String Starts with the Specified String

challenge = 'thirty days of python'
print(challenge.swapcase()) # THIRTY DAYS OF PYTHON
challenge = 'Thirty Days Of Python'
print(challenge.swapcase()) # tHIRTY dAYS oF pYTHON

# startswith(): Checks if String Starts with the Specified String

challenge = 'thirty days of python'
print(challenge.startswith('thirty')) # True
challenge = '30 days of python'
print(challenge.startswith('thirty')) # False
```

```

P
1
Hello, World!
13
I hope you are enjoying 30 days of python challenge
I am a teacher and enjoy teaching.
I didn't find anything as rewarding as empowering people.
That is why I created 30 days of python.
I am a teacher and enjoy teaching.
I didn't find anything as rewarding as empowering people.
That is why I created 30 days of python.
Asabeneh Yetayeh
8
7
True
16
P
y
t
h
o
n
P
y
n
n
o
hon
hon
hon
Pto
I hope every one enjoying the python challenge.
Do you ?
Days    Topics    Exercises
Day 1    3          5
Day 2    3          5
Day 3    3          5
Day 4    3          5
This is a back slash symbol (\)
In every programming language it starts with "Hello, World!"
Thirty days of python
3
1
2
True
False
thirty days of python
thirty days of python
5
0
I am Asabeneh Yetayeh. I am a teacher. I live in Finland.
The area of circle with 10 is 3.14
5
0
True
True

```

```
False
False
False
False
5
0
False
True
False
False
True
True
False
False
True
True
False
HTML#, CSS#, JavaScript#, React
  thirty days of python
thirty days of coding
['thirty', 'days', 'of', 'python']
Thirty Days Of Python
THIRTY DAYS OF PYTHON
tHIRTy dAYS oF pYTHON
True
False
```

In []: